

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра ИУ5 “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №1
“Основные конструкции языка Python”

Выполнил:
Студент группы ИУ5-33Б
Алиева Э. Р.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Цель лабораторной работы

Цель лабораторной работы: изучение основных конструкций языка Python.

Задание лабораторной работы

Разработать программу для решения биквадратного уравнения.

1. Программа должна быть разработана в виде консольного приложения на языке Python.
2. Программа осуществляет ввод с клавиатуры коэффициентов А, В, С, вычисляет дискриминант и **ДЕЙСТВИТЕЛЬНЫЕ** корни уравнения (в зависимости от дискриминанта).
3. Коэффициенты А, В, С могут быть заданы в виде параметров командной строки (вариант задания параметров приведен в конце файла с примером кода). Если они не заданы, то вводятся с клавиатуры в соответствии с пунктом 2. Описание работы с параметрами командной строки.
4. Если коэффициент А, В, С введен или задан в командной строке некорректно, то необходимо проигнорировать некорректное значение и вводить коэффициент повторно пока коэффициент не будет введен корректно. Корректно заданный коэффициент - это коэффициент, значение которого может быть без ошибок преобразовано в действительное число.
5. Дополнительное задание 1 (*). Разработайте две программы на языке Python - одну с применением процедурной парадигмы, а другую с применением объектно-ориентированной парадигмы.
6. Дополнительное задание 2 (*). Разработайте две программы - одну на языке Python, а другую на любом другом языке программирования (кроме C++).

Листинг кода:

biquadratic_equation_func.py

```
import sys
import math

def get_coef(index, prompt):

    try:
        coef_str = sys.argv[index]
        coef = float(coef_str)
    except:
        while(True):
            print(prompt)
            coef_str = input()
            try:
                coef = float(coef_str)
                break
            except:
                print("Введено некорректное значение, введите действительное число")
```

```

    return coef

def get_coefs(coefficients):

    coefficients[0] = get_coef(1, 'Введите коэффициент A:')
    coefficients[1] = get_coef(2, 'Введите коэффициент B:')
    coefficients[2] = get_coef(3, 'Введите коэффициент C:')

    return coefficients

def calculate_roots(coefficients, result):

    num_roots = 0
    roots_list = []

    if coefficients[0] == 0:
        if coefficients[1] == 0:
            if coefficients[2] == 0:
                roots_list = ["Бесконечное множество корней"]
                num_roots = 0
            else:
                roots_list = ["Нет корней"]
                num_roots = 0
        else:
            if coefficients[2] == 0:
                roots_list.append(0)
                num_roots = 1
            else:
                x = (- coefficients[2]) / coefficients[1]
                if x > 0:
                    roots_list.append(math.sqrt(x))
                    roots_list.append(- math.sqrt(x))
                    num_roots = 2
                elif x == 0:
                    roots_list.append(0)
                    num_roots = 1
                else:
                    roots_list = ["Нет корней"]
                    num_roots = 0
    else:
        if coefficients[1] == 0:
            if coefficients[2] == 0:
                roots_list.append(0)
                num_roots = 1
            else:
                fourth_degree = (- coefficients[2]) / coefficients[0]
                if fourth_degree < 0:
                    roots_list = ["Нет корней"]
                    num_roots = 0
                else:
                    x = math.sqrt(fourth_degree)
                    if x == 0:

```

```

        roots_list.append(0)
        num_roots = 1
    else:
        roots_list.append(math.sqrt(x))
        roots_list.append(- math.sqrt(x))
        num_roots = 2
else:
    if coefficients[2] == 0:
        roots_list.append(0)
        num_roots = 1
        x = (- coefficients[1]) / coefficients[0]
        if x > 0:
            roots_list.append(math.sqrt(x))
            roots_list.append(- math.sqrt(x))
            num_roots = 3
    else:
        discriminant = coefficients[1] ** 2 - 4 * coefficients[0] *
coefficients[2]
        if discriminant < 0:
            roots_list = ["Нет корней"]
            num_roots = 0
        elif discriminant == 0:
            t = (- coefficients[1]) / (2 * coefficients[0])
            if t > 0:
                roots_list.append(math.sqrt(t))
                roots_list.append(- math.sqrt(t))
                num_roots = 2
            elif t == 0:
                roots_list.append(0)
                num_roots = 1
            else:
                roots_list = ["Нет корней"]
                num_roots = 0
        else:
            t1 = (- coefficients[1] + math.sqrt(discriminant)) / (2 *
coefficients[0])
            t2 = (- coefficients[1] - math.sqrt(discriminant)) / (2 *
coefficients[0])
            roots_list = []
            num_roots = 0
            if t1 > 0:
                roots_list.append(math.sqrt(t1))
                roots_list.append(- math.sqrt(t1))
                num_roots += 2
            elif t1 == 0:
                roots_list.append(0)
                num_roots += 1
            if t2 > 0:
                roots_list.append(math.sqrt(t2))
                roots_list.append(- math.sqrt(t2))
                num_roots += 2
            elif t2 == 0 and t1 != 0:
                roots_list.append(0)

```

```

        num_roots += 1
        if t1 < 0 and t2 < 0:
            roots_list = ["Нет корней"]
            num_roots = 0

    result = [num_roots, coefficients, roots_list]
    return result

def print_roots(result):
    if len(result[2]) == 1 and isinstance(result[2][0], str):
        if result[0] == 0:
            print(result[2][0])
        else:
            print("Произошла ошибка: несоответствие между количеством корней и
списком корней")
    else:
        if result[0] == len(result[2]):
            if result[0] == 0:
                print("Нет корней")
            else:
                print("Корни:", result[2])
        else:
            print("Произошла ошибка: несоответствие между количеством корней и
списком корней")

def main():
    result = [0, [0.0, 0.0, 0.0], []]
    coefficients = [0.0, 0.0, 0.0]
    coefficients = get_coefs(coefficients)
    result = calculate_roots(coefficients, result)
    print_roots(result)

if __name__ == "__main__":
    main()

```

biquadratic_equation_oop.py

```

import sys
import math

class SquareRoots:

    def __init__(self):

        self.coef_A = 0.0
        self.coef_B = 0.0
        self.coef_C = 0.0
        self.num_roots = 0
        self.roots_list = []

    def get_coef(self, index, prompt):

```

```

try:
    coef_str = sys.argv[index]
    coef = float(coef_str)
except:
    while(True):
        print(prompt)
        coef_str = input()
        try:
            coef = float(coef_str)
            break
        except:
            print("Введено некорректное значение, введите действительное
число")

    return coef

def get_coefs(self):

    self.coef_A = self.get_coef(1, 'Введите коэффициент A:')
    self.coef_B = self.get_coef(2, 'Введите коэффициент B:')
    self.coef_C = self.get_coef(3, 'Введите коэффициент C:')

def calculate_roots(self):

    self.num_roots = 0
    self.roots_list = []

    if self.coef_A == 0:
        if self.coef_B == 0:
            if self.coef_C == 0:
                self.roots_list = ["Бесконечное множество корней"]
                self.num_roots = 0
            else:
                self.roots_list = ["Нет корней"]
                self.num_roots = 0
        else:
            if self.coef_C == 0:
                self.roots_list.append(0)
                self.num_roots = 1
            else:
                x = (- self.coef_C) / self.coef_B
                if x > 0:
                    self.roots_list.append(math.sqrt(x))
                    self.roots_list.append(- math.sqrt(x))
                    self.num_roots = 2
                elif x == 0:
                    self.roots_list.append(0)
                    self.num_roots = 1
                else:
                    self.roots_list = ["Нет корней"]
                    self.num_roots = 0
    else:

```

```

if self.coef_B == 0:
    if self.coef_C == 0:
        self.roots_list.append(0)
        self.num_roots = 1
    else:
        fourth_degree = (- self.coef_C) / self.coef_A
        if fourth_degree < 0:
            self.roots_list = ["Нет корней"]
            self.num_roots = 0
        else:
            x = math.sqrt(fourth_degree)
            if x == 0:
                self.roots_list.append(0)
                self.num_roots = 1
            else:
                self.roots_list.append(math.sqrt(x))
                self.roots_list.append(- math.sqrt(x))
                self.num_roots = 2
else:
    if self.coef_C == 0:
        self.roots_list.append(0)
        self.num_roots = 1
        x = (- self.coef_B) / self.coef_A
        if x > 0:
            self.roots_list.append(math.sqrt(x))
            self.roots_list.append(- math.sqrt(x))
            self.num_roots = 3
    else:
        discriminant = self.coef_B ** 2 - 4 * self.coef_A * self.coef_C
        if discriminant < 0:
            self.roots_list = ["Нет корней"]
            self.num_roots = 0
        elif discriminant == 0:
            t = (- self.coef_B) / (2 * self.coef_A)
            if t > 0:
                self.roots_list.append(math.sqrt(t))
                self.roots_list.append(- math.sqrt(t))
                self.num_roots = 2
            elif t == 0:
                self.roots_list.append(0)
                self.num_roots = 1
            else:
                self.roots_list = ["Нет корней"]
                self.num_roots = 0
        else:
            t1 = (- self.coef_B + math.sqrt(discriminant)) / (2 *
self.coef_A)

            t2 = (- self.coef_B - math.sqrt(discriminant)) / (2 *
self.coef_A)

            self.roots_list = []
            self.num_roots = 0
            if t1 > 0:
                self.roots_list.append(math.sqrt(t1))

```

```

        self.roots_list.append(- math.sqrt(t1))
        self.num_roots += 2
    elif t1 == 0:
        self.roots_list.append(0)
        self.num_roots += 1
    if t2 > 0:
        self.roots_list.append(math.sqrt(t2))
        self.roots_list.append(- math.sqrt(t2))
        self.num_roots += 2
    elif t2 == 0 and t1 != 0:
        self.roots_list.append(0)
        self.num_roots += 1
    if t1 < 0 and t2 < 0:
        self.roots_list = ["Нет корней"]
        self.num_roots = 0

def print_roots(self):
    if len(self.roots_list) == 1 and isinstance(self.roots_list[0], str):
        if self.num_roots == 0:
            print(self.roots_list[0])
        else:
            print("Произошла ошибка: несоответствие между количеством корней и
списком корней")
    else:
        if self.num_roots == len(self.roots_list):
            if self.num_roots == 0:
                print("Нет корней")
            else:
                print("Корни:", self.roots_list)
        else:
            print("Произошла ошибка: несоответствие между количеством корней и
списком корней")

def main():

    r = SquareRoots()
    r.roots_list = []
    r.get_coefs()
    r.calculate_roots()
    r.print_roots()

if __name__ == "__main__":
    main()

```

Скриншот работы программы


```
• (rk2) emiliaaliev@MacBook-Air-Emilia Alieva-PCPL-2025 % /Users/emiliaaliev/Alieva-PCPL-2025/rk2/bin/python /Users/emiliaaliev/Alieva-PCPL-2025/biquadratic_equation_func.py
Введите коэффициент A:
1
Введите коэффициент B:
1
Введите коэффициент C:
-2
Корни: [1.0, -1.0]
• (rk2) emiliaaliev@MacBook-Air-Emilia Alieva-PCPL-2025 % /Users/emiliaaliev/Alieva-PCPL-2025/rk2/bin/python /Users/emiliaaliev/Alieva-PCPL-2025/biquadratic_equation_oop.py
Введите коэффициент A:
1
Введите коэффициент B:
1
Введите коэффициент C:
-2
Корни: [1.0, -1.0]
```